Week 2, Day 5: SQL Indexing, Query Plans, Locking and SQL Practice

Prepared for Upendra Kumar Sharma | 3–4 hours | Main topic: PostgreSQL indexing, EXPLAIN ANALYZE, joins, pagination, locking | Practice: SQL window-function problems

Today's Schedule

- **0:00–0:50** Study indexes and query plans (sections 1 to 3). Run the hands-on exercise in section 3 on a local PostgreSQL.
- **0:50–1:30** Study joins, window functions and pagination (sections 4 to 6).
- **1:30–2:00** Study optimistic and pessimistic locking (section 7).
- **2:00–3:00** SQL practice: solve the 3 problems in section 8, plus the extra problems if time allows.
- **3:00–3:30** Answer the interview questions aloud (section 9) and add 10 flashcards.

This guide uses PostgreSQL, since it is what your resume lists. The ideas carry over to MySQL with small syntax differences.

1. How Indexes Work

- **B-tree (the default):** a balanced tree that keeps keys sorted. Lookups, range scans (<, >, BETWEEN), ORDER BY and prefix searches (LIKE 'abc%') can use it. Cost of a lookup is O(log n).
- **Other index types in PostgreSQL: Hash** (equality only), **GIN** (arrays, jsonb, full-text search), **GiST** (geometric and range data), **BRIN** (very large tables where values correlate with physical order, such as timestamps in an append-only table).
- **What an index costs:** extra disk space, and **slower INSERT, UPDATE and DELETE**, because every index must be maintained. Index what your queries need, not every column.
- **Primary keys and unique constraints** create an index automatically. **Foreign keys do not** in PostgreSQL, so index foreign key columns that you join or filter on, otherwise joins and cascading deletes can be slow.
- **Selectivity:** an index helps most when the condition filters out most rows. An index on a column with three values (such as status) is rarely useful alone, but can be useful as part of a composite or partial index.

2. Composite, Covering, Partial and Expression Indexes

Composite (multi-column) index

- Order matters. An index on (entity_id, status, created_at) can serve queries filtering on entity_id, or entity_id and status, or all three. It generally **cannot** serve a query filtering only on status (the **leftmost prefix rule**).
- **Column order guideline:** equality conditions first, then the range or sort column last.
- An index can also satisfy an ORDER BY if the order matches, which avoids a separate sort step.

Covering index and index-only scans

- If the index contains every column the query needs, PostgreSQL can answer from the index alone (**Index Only Scan**) without visiting the table. Use INCLUDE to add non-key columns: CREATE INDEX ... (entity_id) INCLUDE (status).
- Index-only scans also depend on the visibility map being up to date, which regular vacuuming maintains.

Partial index: indexes only the rows that match a condition. Smaller and faster when you mostly query a subset.

Expression index: indexes the result of a function, so queries using that function can use the index.

```
-- Composite: equality columns first, sort column last
CREATE INDEX idx_complaint_entity_status_created
    ON complaint (entity_id, status, created_at DESC);

-- Partial: only open complaints
CREATE INDEX idx_complaint_open ON complaint (created_at)
    WHERE status = 'OPEN';

-- Expression: supports WHERE lower(email) = 'a@b.com'
CREATE INDEX idx_user_email_lower ON app_user (lower(email));
```

Why an index might not be used

- A function or cast on the column in the WHERE clause (WHERE lower(email) = ... without an expression index, or WHERE created_at::date = ...).
- A leading wildcard: LIKE '%text'.
- A type mismatch that forces an implicit cast.
- The planner estimates that most of the table matches, so a sequential scan is cheaper. This is normal and correct for small tables.
- Stale statistics: run ANALYZE so the planner knows the real data distribution.

- Violating the leftmost prefix rule in a composite index.

3. Reading Query Plans with EXPLAIN ANALYZE

- **EXPLAIN** shows the plan the planner chose, with estimated costs. **EXPLAIN ANALYZE** also **runs** the query and shows actual times and row counts. Add **BUFFERS** to see how much data came from cache vs disk.
- **Caution:** EXPLAIN ANALYZE really executes the statement. For INSERT, UPDATE or DELETE, run it inside a transaction and roll back.

Scan types

- **Seq Scan:** reads the whole table. Fine for small tables or when most rows are needed.
- **Index Scan:** walks the index, then fetches rows from the table.
- **Index Only Scan:** answers from the index alone.
- **Bitmap Index Scan + Bitmap Heap Scan:** collects matching row locations from one or more indexes, then reads table pages in order. Common for medium selectivity.

Join methods

- **Nested Loop:** for each row of the outer side, look up the inner side. Great when the outer side is small and the inner side has an index.
- **Hash Join:** builds a hash table of one side, then probes it. Good for large, unsorted inputs with equality conditions.
- **Merge Join:** merges two sorted inputs. Good when both sides are already sorted, for example by index.

What to look for

- **Estimated rows vs actual rows.** A large mismatch means stale or insufficient statistics, and the planner may choose a bad plan.
- **Which node takes the time.** Look at actual time and the loops count. A fast node running 100,000 loops can dominate.
- **Sort or Hash nodes spilling to disk,** shown in the plan as external or batches greater than one.
- **Rows removed by filter** being huge: an index could help.
- **Buffers:** high shared read means lots of disk access.

Hands-on exercise (do this)

```
CREATE TABLE complaint (
    id bigserial PRIMARY KEY,
    status text NOT NULL,
    entity_id int NOT NULL,
    created_at timestamptz NOT NULL DEFAULT now()
);

INSERT INTO complaint (status, entity_id, created_at)
SELECT (ARRAY['OPEN','CLOSED','PENDING'])[1 + floor(random() * 3)::int],
       (random() * 500)::int,
       now() - (random() * 365) * interval '1 day'
FROM generate_series(1, 1000000);

ANALYZE complaint;

EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM complaint
WHERE entity_id = 42 AND status = 'OPEN'
ORDER BY created_at DESC LIMIT 20;        -- expect a Seq Scan and a Sort

CREATE INDEX idx_complaint_entity_status_created
    ON complaint (entity_id, status, created_at DESC);

EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM complaint
WHERE entity_id = 42 AND status = 'OPEN'
ORDER BY created_at DESC LIMIT 20;        -- expect an Index Scan and no Sort
```

Record the execution time before and after. This before-and-after story, with your own numbers, is exactly what interviewers want when you say you "optimized SQL queries".

4. Joins and Query Patterns

- **INNER JOIN:** only rows that match on both sides.
- **LEFT JOIN:** all rows from the left side, with NULLs where the right side has no match. Putting a condition on the right table in WHERE turns it back into an inner join, so put it in the ON clause.
- **RIGHT and FULL JOIN:** the mirror image, and both sides, respectively. CROSS JOIN is every combination.
- **EXISTS vs IN:** both express semi-joins, and PostgreSQL usually plans them similarly. **Prefer NOT EXISTS over NOT IN**, because NOT IN returns no rows if the subquery contains a NULL.

- **WHERE vs HAVING:** WHERE filters rows before grouping, HAVING filters groups after aggregation.
- **Self join:** joining a table to itself, for example employees to their managers.
- **CTEs (WITH):** make long queries readable. Since PostgreSQL 12, non-recursive CTEs are normally inlined unless marked MATERIALIZED.
- **SELECT only the columns you need,** and avoid SELECT * in application queries.

5. Window Functions

A window function computes a value across a set of related rows **without collapsing them** like GROUP BY does.

- **PARTITION BY** splits rows into groups, **ORDER BY** orders within each group.
- **ROW_NUMBER():** unique sequence 1, 2, 3, even for ties.
- **RANK():** ties share a rank and the next rank is skipped (1, 1, 3).
- **DENSE_RANK():** ties share a rank with no gaps (1, 1, 2).
- **LAG() / LEAD():** value from the previous or next row.
- **SUM() OVER (ORDER BY ...):** running total.

```
SELECT name, department_id, salary,
       ROW_NUMBER() OVER w AS row_num,
       RANK()       OVER w AS rnk,
       DENSE_RANK() OVER w AS dense_rnk
FROM employee
WINDOW w AS (PARTITION BY department_id ORDER BY salary DESC);
```

For salaries 300, 300, 200 in one department, the three columns give 1, 2, 3 for ROW_NUMBER, 1, 1, 3 for RANK, and 1, 1, 2 for DENSE_RANK.

6. Pagination

OFFSET / LIMIT

- Simple, and it is what Spring Data's Pageable uses. But the database must read and discard all the skipped rows, so **deep pages get slower** (page 10,000 is far slower than page 1).
- Results can also shift between requests if rows are inserted or deleted.
- Pageable also runs an extra **count query**. Return a Slice instead of a Page when you do not need the total.

Keyset (seek) pagination

- Instead of an offset, pass the sort key of the last row you returned and ask for rows after it. Performance stays flat for any depth, as long as there is a matching index.
- Needs a **stable, unique sort order**, so include the id as a tie-breaker.
- Cannot jump to an arbitrary page number, so it suits "load more" and infinite scroll APIs. Spring Data JPA 3.1 and later has built-in keyset scrolling support.

```
-- First page
SELECT id, created_at FROM complaint
ORDER BY created_at DESC, id DESC
LIMIT 20;

-- Next page, using the last row's values
SELECT id, created_at FROM complaint
WHERE (created_at, id) < (:lastCreatedAt, :lastId)
ORDER BY created_at DESC, id DESC
LIMIT 20;
```

7. Locking: Optimistic vs Pessimistic

Locking prevents the **lost update** problem: two users read the same row, both change it, and the second save silently overwrites the first.

Optimistic locking

- Add a **version column** (@Version in JPA). Every update becomes UPDATE ... SET ..., version = version + 1 WHERE id = ? AND version = ?. If no row matches, someone else changed it first, and JPA throws OptimisticLockException (Spring translates it to ObjectOptimisticLockingFailureException).
- No database locks are held while the user thinks. Best when **conflicts are rare**, which is typical for web forms and REST APIs.
- On conflict, either **retry** (for automated work) or tell the user the data changed (for interactive work).

```
@Entity
public class Account {
    @Id private Long id;
    private BigDecimal balance;

    @Version
    private long version;
}
```

Pessimistic locking

- Lock the row when you read it: SELECT ... **FOR UPDATE** (JPA: LockModeType.PESSIMISTIC_WRITE). Others who try to lock or change it **wait**.
- Use when **conflicts are frequent** or the cost of a retry is high, for example stock decrement or seat booking. Keep the transaction **short**, because locks are held until commit.
- **NOWAIT** fails immediately instead of waiting. **SKIP LOCKED** skips rows that are already locked, which makes a table usable as a **job queue** with several workers.
- Always lock rows in a **consistent order** to avoid deadlocks. PostgreSQL detects deadlocks and aborts one transaction, so your code must handle that error.

```
public interface AccountRepository extends JpaRepository<Account, Long> {
    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("select a from Account a where a.id = :id")
    Optional<Account> findByIdForUpdate(@Param("id") Long id);
}
```

```
-- Worker picks the next unprocessed job without blocking other workers
SELECT id FROM job
WHERE status = 'PENDING'
ORDER BY id
LIMIT 1
FOR UPDATE SKIP LOCKED;
```

Other techniques

- **Atomic update in one statement:** UPDATE account SET balance = balance - :amt WHERE id = :id AND balance >= :amt, then check the updated row count. Often the simplest and fastest approach.
- **Unique constraints** to enforce idempotency and prevent duplicate inserts.
- **MVCC in PostgreSQL:** readers do not block writers and writers do not block readers, because each transaction sees a snapshot. Only writers to the same row block each other.

8. SQL Practice (3 Problems)

Assume these tables: **employee(id, name, salary, department_id)** and **department(id, name)**.

Problem 1: Second Highest Salary

- **Idea:** the maximum salary among those lower than the overall maximum. It returns NULL when there is no second distinct salary.

- **Edge cases:** only one employee, all salaries equal, duplicate top salaries (the second highest is the next **distinct** value).

```
SELECT MAX(salary) AS second_highest_salary
FROM employee
WHERE salary < (SELECT MAX(salary) FROM employee);

-- Generalizes to the Nth highest with DENSE_RANK
SELECT salary FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employee
) t
WHERE rnk = 2
LIMIT 1;
```

Problem 2: Rank Employees Within Each Department

- **Idea:** use window functions with PARTITION BY department. Write the query from section 5 yourself, then explain out loud how ROW_NUMBER, RANK and DENSE_RANK differ on tied salaries.
- **Follow-up:** add the salary difference from the next-highest colleague in the department using LAG or LEAD.

Problem 3: Department Top Three Salaries

- **Idea:** compute DENSE_RANK per department in a subquery, then keep rank 3 or lower and join to the department for its name. DENSE_RANK is the right choice because "top three salaries" means three **distinct** salary values, and tied employees should all be included.
- **Edge cases:** departments with fewer than three employees, ties at the third value, employees with no department.

```
SELECT d.name AS department, e.name AS employee, e.salary
FROM (
    SELECT name, salary, department_id,
           DENSE_RANK() OVER (PARTITION BY department_id ORDER BY salary
DESC) AS rnk
    FROM employee
) e
JOIN department d ON d.id = e.department_id
WHERE e.rnk <= 3
ORDER BY d.name, e.salary DESC;
```

Extra practice if time allows

- Find duplicate emails: GROUP BY email HAVING COUNT(*) > 1.
- Employees who earn more than their manager: self join on manager_id.
- Customers who never placed an order: NOT EXISTS (or LEFT JOIN ... WHERE o.id IS NULL).
- Running total of daily complaint counts: SUM(count) OVER (ORDER BY day).
- Delete duplicate rows keeping the lowest id.

9. Interview Questions to Answer Aloud

1. How does a B-tree index work, and what does it cost on writes?
2. What is the leftmost prefix rule? How would you order the columns of a composite index?
3. What is a covering index, and what is an index-only scan?
4. Name five reasons the database may ignore an index you created.
5. How do you read EXPLAIN ANALYZE output? What do you check first?
6. When does a sequential scan beat an index scan?
7. Nested loop vs hash join vs merge join: when does each win?
8. Why is NOT EXISTS safer than NOT IN?
9. ROW_NUMBER vs RANK vs DENSE_RANK, with an example.
10. Why does OFFSET pagination get slow on deep pages, and how does keyset pagination fix it?
11. Optimistic vs pessimistic locking: how does each work, and when do you choose each?
12. How would you build a safe job queue on a PostgreSQL table with several workers?
13. Tell me about a query you optimized. What did you measure before and after?

10. End-of-Day Checklist

- [ ] I can explain B-tree, composite, covering, partial and expression indexes with an example of each.
- [ ] I ran EXPLAIN ANALYZE before and after adding an index and noted the difference.
- [ ] I can name the scan and join node types and what to look for in a plan.
- [ ] I can write ROW_NUMBER, RANK and DENSE_RANK queries from memory.
- [ ] I can explain keyset pagination and write the query.
- [ ] I can explain @Version, FOR UPDATE and SKIP LOCKED and when to use each.
- [ ] I solved the 3 SQL problems without looking at the answers.
- [ ] I wrote at least 10 flashcards from questions I got wrong.

Tomorrow (Week 2, Day 6, weekend): Spring Security (filter chain, JWT authentication, OAuth2 basics), REST best practices, exception handling and validation.